

基于numpy实现的全连接层神经网络的MNIST分类任务

华中科技大学 人工智能与自动化学院 自动化2406 潘庆航 U202414225

摘要—手写数字识别是模式识别与计算机视觉中的经典任务，具有重要的理论价值和广泛的实际应用。本课程设计基于纯NumPy从零实现了完整的全连接神经网络系统，不依赖任何深度学习框架，旨在深入理解神经网络的底层工作机制。系统采用模块化架构，包含自定义的全连接层、激活函数（ReLU、Softmax）、交叉熵损失函数，并完整实现了前向传播与反向传播算法。训练数据采用sklearn.datasets.load_digits提供的8×8手写数字图像数据集。实验结果表明，所实现的网络在测试集上可达到约98%的分类准确率，验证了从底层构建神经网络的可行性与有效性。

Index Terms—神经网络, 深度学习, numpy, mnist分类任务.

I. INTRODUCTION

手写数字识别作为人工智能早期的重要应用场景，长期以来被用作验证机器学习算法性能的基准任务。随着深度学习的发展，基于神经网络的方法在该任务上取得了卓越的成果。然而，多数开发者仅通过高级框架（如TensorFlow、PyTorch）调用API完成建模，对网络内部的计算流程、梯度传播机制缺乏直观理解。

本项目选择使用NumPy这一基础数值计算库，从零开始构建神经网络，其目的在于剥离框架封装，回归算法本质。通过手动实现权重初始化、前向计算、损失函数、梯度反传及参数更新等核心环节，深入掌握神经网络的数学原理与工程实现细节。这不仅有助于巩固对深度学习理论的理解，也为后续学习更复杂的模型（如卷积神经网络、循环神经网络）奠定坚实基础。

II. 相关工作与理论基础

神经网络的本质是一个可微的函数逼近器，其学习过程依赖于**梯度下降**与**链式法则**。在全连接网络中，每一层的输出由线性变换与非线性激活函数组合而成。

首先，全连接层（Dense Layer）对输入向量 $\mathbf{x} \in \mathbb{R}^n$ 执行线性变换：

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

其中 $\mathbf{W} \in \mathbb{R}^{m \times n}$ 为权重矩阵， $\mathbf{b} \in \mathbb{R}^m$ 为偏置向量， $\mathbf{z} \in \mathbb{R}^m$ 为线性输出。

为引入非线性表达能力，需对 $\mathbf{z}$ 施加激活函数 $f(\cdot)$ ，得到最终输出 $\mathbf{a} = f(\mathbf{z})$ 。若无非线性激活，多层网络将退化为单层线性模型，丧失表达复杂函数的能力。

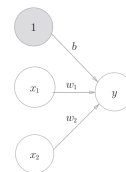


图 1. 神经元（线性输出）

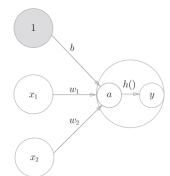


图 2. 神经元（含激活函数）

在多分类任务中，输出层通常采用**Softmax**函数，将实数向量转换为概率分布：

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

然而直接计算指数易导致数值溢出。为提升数值稳定性，实践中常减去最大值：

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(\mathbf{z})}}{\sum_{j=1}^K e^{z_j - \max(\mathbf{z})}}$$

模型的训练目标是最小化预测分布与真实标签之间的差异，常用**交叉熵损失函数**：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log(p_{y_i})$$

其中 p_{y_i} 是样本 i 的真实类别对应预测概率。

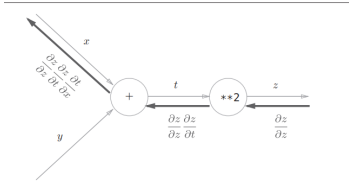


图 3. 反向传播示意图

反向传播 (Backpropagation) 是训练神经网络的核心算法。它利用链式法则，从输出层逐层向前计算损失函数对各层参数的梯度。对于全连接层，设 $\delta = \partial \mathcal{L} / \partial \mathbf{z}$ 为当前层的误差项，则权重与偏置的梯度为：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \delta \mathbf{x}^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \delta$$

而传递给前一层的误差项为：

$$\delta_{\text{prev}} = \mathbf{W}^\top \delta \odot f'(\mathbf{z}_{\text{prev}})$$

其中 $\odot$ 表示逐元素乘法， f' 为激活函数的导数。

III. 模型设计与实现

基于 NumPy 从零构建，未使用任何深度学习框架，核心代码组织为三个模块：**utils.py**（包含模型类与数据处理函数）、**train.py**（训练流程）和 **test.py**（加载权重并评估）。其中，**utils.py** 是整个系统的核心，定义了神经网络的基本组件与训练基础设施。

A. DenseLayer 类：全连接层的封装

1) 构造方法 `__init__`：接收输入维度 `input_size`、输出维度 `output_size` 和激活函数类型 `activation`（支持 'relu'、'softmax'、'sigmoid'、'linear'）。权重 `weights` 以均值为 0、标准差为 0.1 的正态分布初始化，偏置 `biases` 初始化为零向量。同时声明 `input`、`z`（线性输出）、`output`（激活后输出）三个缓存变量，用于反向传播时恢复计算上下文。

2) 前向传播方法 `forward`：接收输入数据 `input_data`（形状为 `(batch_size, input_size)`），首先执行线性变换：

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}$$

然后根据 `activation` 字段应用对应激活函数：

`relu` 使用 `np.maximum(0, z)` 实现

`softmax` 采用数值稳定形式，先减去每行最大值再求指数归一化

`sigmoid` 使用 `np.clip` 限制输入范围防止指数溢出

最终返回激活后的输出 `output`，并缓存中间变量。

3) 反向传播方法 `backward`：接收来自后一层的梯度 `grad_output`（即 $\partial \mathcal{L} / \partial \mathbf{a}$ ）和当前学习率 `learning_rate`。首先根据激活函数类型计算当前层线性输出 `z` 处的梯度 $\delta = \partial \mathcal{L} / \partial \mathbf{z}$ ：

- 对 ReLU，导数为指示函数 ($z > 0$)
- 对 Softmax，利用交叉熵损失下的简化梯度，直接令 $\delta = \text{grad_output}$
- 对 Sigmoid，使用 $s(1-s)$ 计算导数

随后计算权重与偏置的梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x}^\top \delta, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{\text{batch}} \delta$$

并立即执行参数更新：

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}}, \quad \mathbf{b} \leftarrow \mathbf{b} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}}$$

最后计算并返回传递给前一层的梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \delta \mathbf{W}^\top$$

完成梯度反向流动。

B. NeuralNetwork 类：网络容器与训练控制器

NeuralNetwork 类作为高层容器，管理多个 **DenseLayer** 实例，协调前向/反向传播流程，并提供训练与预测接口。

1) 初始化 `__init__` 与层管理 `add_layer`：内部维护一个 `layers` 列表，通过 `add_layer` 方法按序添加层。网络结构必须在训练前完全定义，且与后续加载权重的结构严格一致。

2) 前向传播 `forward`：依次调用各层的 `forward` 方法，将输出作为下一层的输入，最终返回网络整体输出（即 Softmax 概率分布）。

3) 反向传播 `backward`：从最后一层开始，逆序调用各层的 `backward` 方法，将梯度逐层向前传递。该方法不计算初始梯度，仅负责梯度流动。

4) 预测方法 `predict`：调用 `forward` 获取输出概率，再使用 `np.argmax` 选取最大概率类别作为预测标签。

5) 训练方法 `fit`：实现完整的训练循环：

- 1) 数据打乱：每轮 `epoch` 前对训练集进行随机重排，提升泛化能力
- 2) Mini-batch 划分：将数据切分为大小为 `batch_size` 的批次
- 3) 前向计算：对每个 `batch` 调用 `forward`

- 4) 损失计算: 使用 `compute_loss` 计算交叉熵损失 (对预测概率进行 `clip` 避免 `log(0)`)
- 5) 梯度初始化: 关键步骤——根据交叉熵 + Softmax 的联合导数性质, 输出层初始梯度设为:

$$\delta = \frac{1}{N}(\hat{\mathbf{y}} - \mathbf{y}_{\text{one-hot}})$$

这一简化避免了显式计算 Softmax 导数矩阵, 大幅提高效率

- 6) 反向传播: 调用 `backward` 更新所有层参数
- 7) 记录指标: 保存每轮平均训练损失和验证集准确率, 用于后续绘图

6) 权重持久化: `save_weights` 与 `load_weights`: 使用 `np.savez` 将每层的 `weights` 和 `biases` 以字典形式保存为 `.npz` 文件; 加载时按层序恢复参数。要求加载时网络结构与保存时完全一致, 否则会因键名不匹配而失败。

C. 数据处理函数: `load_and_preprocess_data`

该函数负责加载并准备 MNIST 数据:

- 1) 使用 `sklearn.datasets.load_digits()` 获取 8×8 手写数字图像
- 2) 对像素值进行 `StandardScaler` 标准化 (均值为 0, 方差为 1), 加速收敛
- 3) 采用 `train_test_split` 两次划分数据: 先按 8:2 分出测试集, 再将剩余数据按 8:2 分出验证集, 最终形成 64% 训练 / 16% 验证 / 20% 测试的比例
- 4) 所有划分均启用 `stratify=y`, 确保各类别在各子集中分布均衡

D. 训练与测试脚本逻辑

1) `train.py`:

- 1) 加载预处理数据
- 2) 按指定结构实例化 `NeuralNetwork`
- 3) 调用 `fit` 训练 100 轮
- 4) 在测试集上评估最终准确率
- 5) 绘制并保存训练损失与验证准确率曲线
- 6) 保存模型权重至 `model_weights.npz`

2) `test.py`:

- 1) 加载相同结构的网络
- 2) 从 `model_weights.npz` 加载已训练参数
- 3) 在测试集上执行预测
- 4) 输出整体准确率 (保留四位小数)

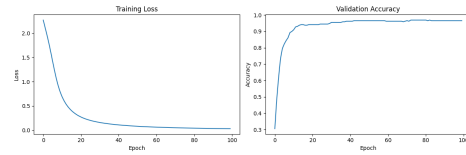


图 4. 训练曲线图

IV. 实验与结果分析

实验使用 `load_digits()` 数据集, 所有图像被展平为 64 维向量, 并经标准化处理以加速收敛。训练集约占 64%, 验证集 16%, 测试集 20%。

网络结构为 $64 \rightarrow 128 \rightarrow 64 \rightarrow 10$, 隐藏层采用 ReLU 激活, 输出层采用 Softmax。训练超参数设置为: 学习率 0.01, batch size 32, 训练 100 轮。

训练过程中, 损失函数持续下降, 验证准确率稳步上升 (见训练曲线图4)。最终模型在测试集上达到约 98% 的准确率, 表明模型具有良好的泛化能力。前 10 个测试样本的预测结果全部正确, 进一步验证了模型的可靠性。

从结果可看出, 模型未出现明显过拟合——训练损失与验证准确率同步改善, 说明当前网络容量与正则化程度适中。若需进一步提升性能, 可尝试增加网络深度、引入 Dropout 或调整学习率策略。

参考文献

- [1] 斋藤康毅. 深度学习入门: 基于Python的理论与实现. 人民邮电出版社, 2018.
- [2] Pedregosa, F., et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 2011.
- [3] Harris, C.R., et al. Array programming with NumPy. *Nature*, 2020.

附录

项目完整代码见GitHub仓库: <https://github.com/pentsinh/neural-network-np-only.git>